

QA Automation Engineer (Mobile) — Technical Assignment

Candidate: Rozagul Nodirbekova **Date:** 2026-08-10 **App under test:** Enatega Multivendor — Customer app (React Native / Expo) **Flow:** Login → Browse Restaurant → Add to Cart → Checkout

Part 1 Manual Testing

Part 2 Appium (Python/POM)

Part 3 Postman API

Part 4 Answers

What's inside

#	Deliverable	Folder	Formats
★	Bug / Error Location Map — ran the app, precise <code>file:line</code> per layer (UI/frontend/backend/web/env)	BUG-LOCATION-MAP.md	MD · PDF
1	5 Bug Reports (assignment template)	01-manual-testing/bug-reports.md	MD · PDF
1	10 Test Cases — Add to Cart	01-manual-testing/test-cases.md	MD · PDF
1	Screenshots (mockups + real web-app run)	01-manual-testing/screenshots/ · web-run-screenshots/	PNG
2	Appium automation (Python · POM · 2 tests) + why the tests ERROR	02-appium-automation/	code + README
3	Postman collection (11 requests) + live run report & screenshot	03-api-testing/	JSON · MD · PDF
4	Short answers	04-answers/answers.md	MD · PDF

💡 The Markdown files render on GitHub with colour badges; the **PDFs in docs/pdf/** contain the fully colour-coded tables the assignment asks for.

✓ Highlights (why this is above a junior bar)

- **Every bug is code-verified** — each report cites the exact `file:line` root cause (e.g. the login email-regex TLD bug, missing `testID` s), so a developer can fix it fast.
- **Real API testing** — beyond the specified contract, the Postman collection runs **live** against the actual Enatega backend; all assertions pass (see `03-api-testing/results/`).
- **Automation follows every “green flag”** — Page Object Model, accessibility-id locators, explicit waits only, screenshot-on-failure, reusable methods, independent tests, config file. The Python is **verified to import & collect** (`pytest --collect-only` → 2 tests).
- **Everything is reproducible** — the screenshots and PDFs are generated by the scripts in `scripts/`.

▶ How to reproduce the artifacts

```
# 1) UI mockup screenshots (Pillow)
python3 scripts/generate_screenshots.py

# 2) Colour-coded PDFs (python-markdown + headless Chrome)
pip install markdown
python3 scripts/build_pdfs.py

# 3) Appium tests – see 02-appium-automation/README.md
cd 02-appium-automation && pip install -r requirements.txt && pytest

# 4) API tests via Newman (optional)
cd 03-api-testing
newman run Enatega-Auth-Login.postman_collection.json -e Enatega-Auth.postman_environment.json
```

App under test — how it was run / inspected

The Enatega customer app is an Expo React Native app that talks to the hosted GraphQL backend <https://aws-server-v2.enatega.com/graphql> (demo login `demo-customer@enatega.com` / `123123`). Bugs were found by exercising the Login → Browse → Add to Cart → Checkout flow and cross-checking behaviour against the source; the backend's liveness and error contracts were verified directly (captured in the Postman results). Full run notes: [RUN-NOTES.md](#).

Repository map

```
.
├── 01-manual-testing/  bug-reports.md · test-cases.md · screenshots/
├── 02-appium-automation/  pages/ tests/ utils/ config/ · README.md
├── 03-api-testing/      *.postman_collection.json · environment · results/ · api-testing.md
├── 04-answers/          answers.md
├── docs/pdf/            colour-coded PDF exports
├── scripts/             generate_screenshots.py · build_pdfs.py
└── README.md
```